

Smart Autonomous Grass Cutting and Weedicide Rover

Project Type	Embedded Robotics / Automation
Course Code:-	DM104
Course Instructor:-	Dr. Anuj Kumar
Prepared By	Vikram Singh(125AD0005) Akshat Gupta(125CS0001) Monika Sharma(525EC0001) Devanshu Dev(525EC0002) Anusha Sinha(125ME0011)
Date	March 2026

1. EXECUTIVE SUMMARY

This report presents a comprehensive technical overview of an autonomous, obstacle-avoiding robot integrated with a glass-cutting mechanism, built around the Arduino Uno microcontroller platform. The system combines ultrasonic distance sensing, servo-controlled directional scanning, dual-channel DC motor drive, and an independently actuated cutting motor to achieve fully automated navigation and material processing.

The robot continuously measures frontal clearance. When an obstruction is detected within a 15cm threshold, it halts, scans left and right using a servo-mounted ultrasonic sensor, selects the clearer path, executes a timed turn, and resumes forward motion. The glass-cutter motor is activated only during forward movement, ensuring operational safety.

All hardware interfaces, pin assignments, firmware logic, and safety considerations are documented herein to support manufacturing, testing, and future development.

2. SYSTEM OVERVIEW

2.1 Functional Description

The robot operates in a continuous feedback loop: sense → decide → act. An HC-SR04 ultrasonic sensor mounted on a servo motor provides 180-degree spatial awareness. Two DC motors (left and right drive) are controlled via an L298N H-bridge driver. A third motor or actuator (the glass cutter) is toggled through a relay or transistor switch on digital pin 7.

2.2 Control Flow Summary

-
- Servo centers to 90° (forward scan).
 - Measure frontal distance via HC-SR04.
 - If distance > 15 cm: move forward, activate cutter.
 - If distance ≤ 15 cm: stop, scan left (150°), scan right (30°).
 - Turn toward the side with greater clearance for ~600 ms.
 - Return to top of loop.

3. COMPONENT DETAILS

3.1 Arduino Uno (ATmega328P)

The Arduino Uno is the central processing unit of the robot. It runs the main control firmware, reads sensor data, and drives all output peripherals.

Specification	Value
Microcontroller	ATmega328P
Operating Voltage	5 V
Input Voltage	7 – 12 V (Vin)
Digital I/O Pins	14 (6 PWM)
Analog Input Pins	6
Flash Memory	32 KB (0.5 KB bootloader)
SRAM	2 KB
Clock Speed	16 MHz

3.2 HC-SR04 Ultrasonic Sensor

The HC-SR04 measures distance by emitting a 40 kHz ultrasonic burst and timing the echo return. The firmware triggers a 10 μ s HIGH pulse on the Trig pin, then measures the echo pulse width with `pulseIn()` to derive distance: $\text{distance} = \text{duration} \times 0.034 / 2$ (cm).

Key specifications:

- **Power:** 5 V DC supply required
- **Range:** 2 – 400 cm (optimum: 2 – 200 cm)
- **Accuracy:** ± 3 mm
- **Beam Angle:** 15° effective cone angle
- **Current Draw:** ~ 15 mA
- **Measurement Cycle:** 40 ms minimum between readings

3.3 Servo Motor (Generic 9g / SG90-class)

A standard PWM servo rotates the ultrasonic sensor for left/right scanning. The Arduino Servo library generates the required 50 Hz PWM signal (pulse width 1–2 ms $\rightarrow$ 0–180°). The sensor is swept to 150° (left) and 30° (right) during obstacle evaluation, with a 700 ms settle delay at each position.

- **Operating Voltage:** 4.8 – 6.0 V
- **Rotation Range:** 0° – 180°
- **Torque:** ~ 1.5 kg·cm at 5 V
- **Current:** ~ 100 – 150 mA peak

- **Arduino Pin:** Pin 4 (Signal)

3.4 L298N Dual H-Bridge Motor Driver

The L298N module drives two DC motors independently using an H-bridge topology that allows bidirectional control. PWM signals on ENA/ENB pins set motor speed (0–255); IN1–IN4 logic levels determine direction. The onboard 5 V regulator can power the Arduino Uno via Vin if the jumper is installed and supply voltage is within range.

Pin mapping in firmware:

Arduino Pin	L298N Pin	Function
5 (PWM)	ENA	Left motor speed control
6 (PWM)	ENB	Right motor speed control
8	IN1	Left motor direction A
9	IN2	Left motor direction B
10	IN3	Right motor direction A
11	IN4	Right motor direction B
GND	GND	Common ground

Motor direction truth table:

IN1 / IN3	IN2 / IN4	Motor Action
HIGH	LOW	Forward
LOW	HIGH	Reverse
LOW	LOW	Stop (coast)

3.5 Glass Cutter Motor / Actuator

Pin 7 drives a relay module or NPN transistor (e.g., 2N2222 / TIP120) that switches power to the cutting motor. The cutter is enabled exclusively during forward motion (moveForward()) and is de-energised whenever the robot stops or turns, reducing wear and hazard risk.

⚠ **SAFETY NOTICE:** Digital output pins supply a maximum of 40 mA at 5 V. Never connect the cutter motor directly to pin 7. Always use an intermediate switching circuit (relay module or transistor with flyback diode) rated for the motor's stall current.

3.6 12V Diaphragm Pump

A 12V diaphragm pump is a compact and efficient device used for transferring liquids such as water, chemicals, or fuel. It operates on a 12-volt DC power supply and provides consistent pressure with self-priming capability. The pump is designed for low power consumption, durability, and smooth operation with minimal noise. It is widely used in agricultural spraying systems, water circulation, and portable fluid transfer applications.

The pump is connected directly to a 12V battery by joining the positive terminal of the pump to the positive terminal of the battery and the negative terminal to the negative terminal. A switch is connected in series with the circuit to control the ON/OFF operation of the pump.

Connection:

The pump is connected directly to a 12V battery by joining the positive terminal of the pump to the positive terminal of the battery and the negative terminal to the negative terminal. A switch is connected in series with the circuit to control the ON/OFF operation of the pump.

3.7 Power Supply

The recommended power architecture separates logic and motor supplies to prevent voltage droops on the Arduino during motor switching.

- **Motor Supply:** 7.4 V – 12 V Li-ion / NiMH pack (minimum 2 A rated)
- **Arduino Power:** L298N 5 V output via Vin jumper, or separate 5 V USB powerbank
- **Servo Power:** Servo powered from Arduino 5 V pin (ensure total draw < 500 mA)

4. PIN CONNECTION REFERENCE

The table below consolidates every electrical connection required to assemble the robot. All connections reference the Arduino Uno as the central node.

Arduino Pin	Connected To	Module	Notes
2 (Digital)	Trig	HC-SR04	10 μ s trigger pulse output
3 (Digital)	Echo	HC-SR04	Pulse-width input
4 (Digital)	Signal (PWM)	Servo Motor	50 Hz PWM via Servo library

5 (PWM)	ENA	L298N	Left motor speed (analogWrite)
6 (PWM)	ENB	L298N	Right motor speed (analogWrite)
8 (Digital)	IN1	L298N	Left motor direction A
9 (Digital)	IN2	L298N	Left motor direction B
10 (Digital)	IN3	L298N	Right motor direction A
11 (Digital)	IN4	L298N	Right motor direction B (also PWM)
5 V	VCC	HC-SR04	5 V regulated supply
5 V	VCC (Red wire)	Servo Motor	< 500 mA total on 5 V rail
GND	GND	HC-SR04	Common ground
GND	GND (Black wire)	Servo Motor	Common ground
GND	GND	L298N	Common ground with motor supply
Vin	5 V Output (jumper)	L298N	Arduino power from L298N onboard reg

5. FIRMWARE ARCHITECTURE

5.1 Initialization (setup())

Serial communication is started at 9600 baud for debug output. All motor driver and cutter pins are configured as OUTPUT; ultrasonic Trig as OUTPUT and Echo as INPUT. The servo is attached and centered to 90° with a 1-second stabilization delay.

5.2 Distance Measurement (getDistance())

A 2 µs LOW pre-pulse clears residual Trig state. A 10 µs HIGH pulse initiates the ultrasonic burst. pulseIn(echoPin, HIGH) measures the echo return time in microseconds. Distance is derived from the formula:

$$\text{distance (cm)} = \text{duration (}\mu\text{s)} \times 0.034 / 2$$

5.3 Motion Functions

Four motion primitives are implemented:

- **moveForward():** ENA/ENB = 180/255, IN1=HIGH, IN2=LOW, IN3=HIGH, IN4=LOW. Cutter ON.

- **stopMotors():** All IN pins LOW. Cutter OFF.
- **turnLeft():** ENA/ENB = 150/255; IN1=LOW, IN2=HIGH, IN3=HIGH, IN4=LOW.
- **turnRight():** ENA/ENB = 150/255; IN1=HIGH, IN2=LOW, IN3=LOW, IN4=HIGH.

5.4 Main Loop (loop())

Each iteration centers the servo, waits 500 ms for mechanical settle, then reads front distance. If clear (> 20 cm) the robot moves forward. On detection, it stops, scans both sides (700 ms settle each), picks the direction with greater clearance, turns for 600 ms, and stops before the next iteration begins.

6. SAFETY & DESIGN CONSIDERATIONS

6.1 Pin 11 PWM Conflict Note

Pin 11 on the Arduino Uno is a hardware PWM pin (Timer 2). In this design it is used as a plain digital output for IN4 with no conflict. If future versions require additional PWM channels, reassign IN4 to a non-PWM digital pin (12 or 13) to free pin 11.

6.2 Cutter Motor Isolation

The cutting motor must not be wired directly to pin 7. A relay module or NPN transistor circuit with a flyback protection diode (e.g., 1N4007) is mandatory. This protects the ATmega328P from back-EMF and current spikes generated by inductive motor loads.

6.3 Power Rail Management

The Arduino 5 V rail shared between the HC-SR04 and servo has a ~500 mA budget. The servo can draw up to 250 mA under load. If additional servos or sensors are added, an external 5 V regulator is recommended for peripheral power.

6.4 Obstacle Detection Timing

At maximum forward speed (PWM 180/255 $\approx$ 70%) a typical robot chassis may travel 10–15 cm during the 500 ms servo-settle delay. Reduce this delay or increase the threshold distance for high-speed applications.

7. BILL OF MATERIALS (BOM)

Qty	Component	Part / Model	Notes
1	Microcontroller Board	Arduino Uno R3	ATmega328P, 5 V logic
1	Ultrasonic Distance Sensor	HC-SR04	2–400 cm range
1	Servo Motor	SG90 or MG90S	9g, 180° rotation
1	Dual H-Bridge Motor Driver	L298N Module	Max 2 A per channel
2	DC Gear Motor	Generic 6–12 V	Left and right drive
1	Glass Cutter Motor / Spindle	As specified	Via relay or transistor
1	12V Diaphragm Pump	As specified	Connected to battery
1	Battery Pack	7.4 V – 12 V	Min 2 Ah capacity
—	Jumper Wires, PCB / Chassis	—	As required

8. FIRMWARE LISTING

Complete Arduino source code (main sketch):

```
#include <Servo.h>
Servo myServo;

// Ultrasonic pins
#define trigPin 2
#define echoPin 3

// Motor driver pins
#define ENA 5
#define IN1 8
#define IN2 9
#define IN3 10
#define IN4 11
#define ENB 6

// Servo & cutter pins
#define servoPin 4
#define cutterPin 7

long duration;
int distance;

void setup() {
  Serial.begin(9600);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  pinMode(ENA, OUTPUT);
  pinMode(ENB, OUTPUT);
}
```



```

pinMode(IN1,      OUTPUT);
pinMode(IN2,      OUTPUT);
pinMode(IN3,      OUTPUT);
pinMode(IN4,      OUTPUT);
pinMode(cutterPin,OUTPUT);
myServo.attach(servoPin);
myServo.write(90);    // Center position
delay(1000);
}

int getDistance() {
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);
    duration = pulseIn(echoPin, HIGH);
    distance = duration * 0.034 / 2;
    return distance;
}

void moveForward() {
    analogWrite(ENA, 180);
    analogWrite(ENB, 180);
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, HIGH);
    digitalWrite(IN4, LOW);
    digitalWrite(cutterPin, HIGH);    // Cutter ON
}

void stopMotors() {
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, LOW);
    digitalWrite(cutterPin, LOW);    // Cutter OFF
}

void turnLeft() {
    analogWrite(ENA, 150);
    analogWrite(ENB, 150);
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);
    digitalWrite(IN3, HIGH);
    digitalWrite(IN4, LOW);
}

void turnRight() {
    analogWrite(ENA, 150);
    analogWrite(ENB, 150);
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, HIGH);
}

void loop() {

```

```
myServo.write(90);
delay(500);
int frontDistance = getDistance();

if (frontDistance > 20) {
    moveForward();
} else {
    stopMotors();
    delay(500);

    myServo.write(150);          // Look Left
    delay(700);
    int leftDistance = getDistance();

    myServo.write(30);           // Look Right
    delay(700);
    int rightDistance = getDistance();

    myServo.write(90);
    delay(500);

    if (leftDistance > rightDistance) {
        turnLeft();
        delay(600);
    } else {
        turnRight();
        delay(600);
    }
    stopMotors();
}
}
```